

Sistemes Operatius (SO)

## PRÀCTICA 4

Mario VILAR  
20392724

David Díez  
20392643

26 de maig de 2023



UNIVERSITAT DE  
BARCELONA

### ÍNDEX

|          |                                 |          |                               |          |
|----------|---------------------------------|----------|-------------------------------|----------|
| <b>1</b> | <b>Introducción</b>             | <b>2</b> | <b>Observació 2</b> . . . . . | <b>4</b> |
|          | <i>Exercici 1</i> . . . . .     | 2        | <i>Figura 2.3</i> . . . . .   | 4        |
| <b>2</b> | <b>Implementación y pruebas</b> | <b>3</b> | <i>Observació 3</i> . . . . . | 4        |
|          | <i>Codi 2.1</i> . . . . .       | 3        | <i>Observació 4</i> . . . . . | 5        |
|          | <i>Codi 2.2</i> . . . . .       | 3        | <i>Codi 2.3</i> . . . . .     | 5        |
|          | <i>Figura 2.1</i> . . . . .     | 3        |                               |          |
|          | <i>Figura 2.2</i> . . . . .     | 4        | <b>3 Conclusiones</b>         | <b>5</b> |

## INTRODUCCIÓN

El objetivo de esta práctica es realizar una implementación propia de la gestión de memoria dinámica en un proceso a través de la implementación de las funciones de la librería estándar malloc y free. Estas dos funciones no son llamadas a sistema, sino que son llamadas que forman parte de la librería de usuario. Internamente utilizan llamadas a sistema.

La firma de la función malloc (memory allocation) de C es la siguiente: `void *malloc(size_t size)`. Como parámetro de entrada recibe un número de bytes a reservar y devuelve un apuntador al bloques de datos que se ha reservado. Una forma de implementar el malloc es utilizando la llamada a sistema sbrk, una función que permite manipular el espacio de heap del proceso.

### Exercici I.

1. Implementación de la función `free(void ptr)` que utilice la estructura propuesta. Pare implementar esta función hay que tener en cuenta que se pasa como parámetro a la función free un puntero en los datos (la variable `ptr` del dibujo), pero la estructura se encuentra “justo debajo”. Básicamente la función free debe poner el atributo `available` a 1. Para asegurar que la función hace el trabajo correctamente, compruebe que el atributo `magic` tiene el valor que debe tener (en caso contrario, imprima un mensaje de error). Hay que tener en cuenta que se puede llamar a free con un apuntador en NULL. En este caso debe ignorarse la llamada. Imprime por pantalla el valor de `size_bytes` que se libera.
2. Una vez implementada la función free asegurarse de que funciona correctamente. Pruebe la su implementación utilizando el código `exemple.c` que se muestra al inicio de la práctica.
3. Implementar la función `void *calloc(size_t nelem, size_t elsize)`. La función calloc permite reservar varios elementos de memoria, en concreto `nelem` elementos de tamaño `elsize` bytes, y los deja inicializados a cero. Se aconseja utilizar la función `memset` para inicializar el bloque de datos a cero. La función devuelve un puntero a la memoria reservada.
4. Implementar la función `void realloc(void ptr, size_t size_bytes)`. La función realloc reajusta el tamaño de un bloque de memoria obtenido con malloc a un nuevo tamaño. Se propone que la implementación de la función realloc sea la siguiente: a) si le pasamos un puntero NULL en `ptr`, se supone que la función realloc actúa como un malloc normal y corriente. b) si le pasamos a `ptr` un apuntador que hemos creado con nuestro malloc y el tamaño que pedimos es suficiente con el bloque que ya tiene reservado, no hace falta hacer nada, lo devolvemos el puntero tal cual. c) en caso contrario, deberemos reservar un nuevo bloque con más espacio y copiar los datos del antiguo bloque en este nuevo. Por eso se puede usar la función `memcpy` para copiar el contenido de un bloque en otro.

## IMPLEMENTACIÓN Y PRUEBAS

Hemos creado dos scripts Bash donde pondremos a prueba el `grep` y el `find` como se nos solicita. El código tanto del `find` como el `grep` son los siguientes:

```
1  #!/bin/bash
2
3  gcc -O -shared -fPIC $1.c -o $1.so
4  export LD_PRELOAD=$PWD/$1.so
5  find files
```

Codi 2.1: Nuestro script *exec\_find.sh*.

```
1  #!/bin/bash
2
3  gcc -O -shared -fPIC $1.c -o $1.so
4  export LD_PRELOAD=$PWD/$1.so
5  grep "Frankenstein" PhineasyFerb.txt
```

Codi 2.2: Nuestro script *exec\_grep.sh*.

Figura 2.1: Se puede probarlos ejecutándolos rutinariamente: `./exec_find.sh malloc_best_fit`, por ejemplo, en el caso que queramos probar el `find` para el `malloc_best_fit`.

Asimismo, no hemos tocado nada del código `exemple.c`, que también nos ha servido como prueba. Explicaremos sucintamente algunas de las funciones que utilizamos. No nos extenderemos porque **el código fuente está sobradamente comentado**.

1. La función `search_available_space` se utiliza para encontrar un bloque de memoria libre de un tamaño especificado. Recorre la lista enlazada de bloques asignados y verifica si el bloque está disponible y tiene el tamaño adecuado.
2. La función `request_space` se utiliza para solicitar un nuevo bloque de memoria del sistema operativo. Utiliza la función `sbrk` para solicitar espacio adicional en el montón y luego configura los metadatos del bloque asignado.
3. La función `free` se utiliza para liberar un bloque de memoria previamente asignado. Verifica si el puntero pasado como argumento es nulo, ajusta el puntero para tener en cuenta los metadatos y marca el bloque como disponible.
4. La función `malloc` se utiliza para asignar un nuevo bloque de memoria del tamaño especificado. Verifica si el tamaño solicitado es válido y un múltiplo de 8. Luego, busca un bloque de memoria libre utilizando la función `search_available_space`. Si encuentra un bloque libre, lo marca como ocupado. Si no encuentra un bloque libre, solicita uno nuevo utilizando la función `request_space` y lo agrega a la lista enlazada de bloques asignados.
5. La función `calloc` se utiliza para asignar y inicializar un bloque de memoria del tamaño especificado con ceros. Verifica si los tamaños de los elementos y el tamaño total son válidos. Luego, utiliza la función `malloc` para asignar el bloque de memoria y la función `memset` para inicializarlo con ceros.

6. La función `realloc` se utiliza para cambiar el tamaño de un bloque de memoria previamente asignado. Verifica si el puntero pasado como argumento es nulo y realiza diferentes acciones dependiendo del caso. Si el puntero es `NULL`, actúa como `malloc`. Si la nueva medida es 0 o negativa, libera el espacio y retorna `NULL`. Luego, verifica la integridad del bloque y ajusta la nueva medida. Si la medida actual es suficiente, retorna el puntero ajustado. Si no, asigna un nuevo bloque, copia los datos y libera el espacio del bloque original. Retorna el puntero del nuevo bloque.

|                                 | <code>exemple.c</code> | <code>exec_find.sh</code>                               | <code>exec_grep.sh</code>    |
|---------------------------------|------------------------|---------------------------------------------------------|------------------------------|
| <code>malloc_first_fit.c</code> | 0,1,2,3,4,5,6,7,8,9    | files<br>files/JohnnyTest.txt<br>files/PhineasyFerb.txt | <i>sesos de frankenstein</i> |
| <code>malloc_best_fit.c</code>  | 0,1,2,3,4,5,6,7,8,9    | files<br>files/JohnnyTest.txt<br>files/PhineasyFerb.txt | <i>sesos de frankenstein</i> |

Figura 2.2: Tabla para clasificar el *malloc* y las salidas recibidas en openSUSE.

**Observació 2** (Salida por terminal de los programas). Claro está que hemos cogido lo más esencial de las salidas para demostrar que nuestro programa funciona. Adjuntaremos el nombre de cada una de ellas en la tabla siguiente, 2.3.

|                                 | <code>exemple.c</code> | <code>exec_find.sh</code> | <code>exec_grep.sh</code> |
|---------------------------------|------------------------|---------------------------|---------------------------|
| <code>malloc_first_fit.c</code> | 1.txt                  | 2.txt                     | 3.txt                     |
| <code>malloc_best_fit.c</code>  | 4.txt                  | 5.txt                     | 6.txt                     |

Figura 2.3: Relación entre *malloc*, programa ejecutado y salida producida en openSUSE.

Podremos encontrar estos ficheros en el directorio de la práctica out. 

Se debe tener en cuenta que ya hemos creado un directorio con dos ficheros en formato txt, PhineasyFerb y JohnnyText. Con esta sencilla estructura será suficiente para testear nuestro find. A su vez, tenemos el mismo PhineasyFerb copiado al mismo nivel de carpeta que los *scripts*, para poder ejecutar sin ningún tipo de problema el find de la palabra *frankenstein*.

**Observació 3** (Implementación opcional). Como comentaremos en la conclusión, el tiempo no nos ha ayudado, dado que necesitábamos de recursos que no estaban a nuestro alcance con la frecuencia que deseábamos. Hemos hecho una implementación del `malloc_opcional.c` y nos gustaría que fuera valorada, aunque, siendo honestos, no hemos tenido la posibilidad de testearla en una máquina con una distribución Linux *clásica* (sí en openSUSE, pero no en Debian o en Ubuntu<sup>1</sup>). 

<sup>1</sup> Esto es relevante porque hemos testeado dos códigos idénticos nos han dado diferentes resultados en estas dos primeras distribuciones.

**Observació 4** (*Exhausted memory*). También hemos tenido problemas con el mensaje *exhausted memory*. En la sintonía de la observación 3 y la nota al pie correspondiente, depende de qué distribución de Linux estemos usando, la salida y la gestión interna de los recursos será una u otra. **Hemos adjuntado, a parte de las salidas obtenidas en openSUSE, en Debian. Las distinguiremos por el sufijo \_D.**

```

1 while(aux != NULL) {
2     if(aux->available == 1) { // si el bloc està lliure
3         aux->previous = prev; // prev apunta a ptr inicialment
4         aux->previous->next = aux->next;
5         if(aux->next != NULL) aux->next->previous = aux->previous; // canviem alguns
        ↪ punters
6         aux->previous->size_bytes += aux->size_bytes; // sumem la mida en bytes del bloc
7         acum += aux->size_bytes;
8     } else break;
9     prev = aux; aux = aux->next; // assignació de punters
10 }
11
12 aux = ptr; aux = aux->previous; // assignació de punters
13
14 while(aux != NULL) { // procediment anàleg, ara per darrere
15     if(aux->available == 1) {
16         aux->next = nex;
17         aux->next->previous = aux->previous;
18         if(aux->previous != NULL) aux->previous->next = aux->next;
19         aux->next->size_bytes += aux->size_bytes;
20         acum += aux->size_bytes;
21     } else break;
22     nex = aux; aux = aux->previous; // assignació de punters
23 }

```

Codi 2.3: Alternativa para juntar bloques libres.

3

## CONCLUSIONES

Por desgracia, no hemos tenido todo el tiempo que hubiéramos deseado en esta práctica, ya que se requería de Linux y ninguno de los dos tenemos acceso excepto en los equipos de la universidad. Luego, en cierta manera, hemos tenido que ser más eficientes para llegar a buen puerto en el menor tiempo posible. Ha sido una experiencia un tanto distinta pero que nos ha ayudado a trabajar de otra manera a la que no estamos acostumbrados.